In this chapter, we will analyze a particular sub-class of containers, strings containers, and investigate how we can optimize them both with respect to memory and running time, introducing new specialized data structures: tries and radix tries.; then we'll use those containers to implement an efficient spell-checker.

6.1 Spell Check

But first, let's introduce the problem that we will solve in this chapter: spell checking.

Well, not that the problem per-se really needs any introduction, right? We would like to have a piece of software that can take words as inputs and return `true` or `false`, respectively if the input is a valid English word[4], or not.

That's a bit of a vague description though, that leaves the door open to many possible (and possibly inefficient) solutions; we need more context to clarify our requirements: let's assume that we are developing a new client for a social network, on a client with low resources (as it might be, for instance, a mobile OS) and we need to add a live spell checker that produces the classic red wavy underline below a misspelled word; every time we type a word (so, every time we type a word separator, like space, commas etc…) we need to check if there is a typo.

Due to the scarce resources, we need our spell-checker to be fast, and light-weight: we need to reduce its impact as much as possible, both in terms of CPU and memory usage.